STUDENT ATTENDANCE MANAGEMENT SYSTEM

A PROJECT REPORT
Submitted for the Subject
OPEN SOURCE DATABASE MANAGEMENT SYSTEM
(OSDBMS)

Submitted By:
Janhavi Amberkar

Academic Year:
2025– 2026

Established – 1961 Subject: OSDBMS

SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE

ULHASNAGAR – 421 003

CERTIFICATE

This is to certify that Mr./Ms. Janhavi Pradip Amberkar of S.Y. Information Technology (SYIT) Roll No. 2542001 has satisfactorily completed the Open Source DataBase Management System Mini Project entitled Student Attendance Management System During the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE **HEAD OF DEPT**

_____ _____

Acknowledgement:

I would like to express my sincere gratitude to the University of Mumbai for providing the academic platform and curriculum that encouraged me to undertake this project on Open Source Database Management Systems using MySQL.

I am deeply thankful to my respected guide, Asst. Prof. Sahil Shukla, for his valuable guidance, continuous support, and constructive suggestions throughout the development of this project titled “Student Attendance Management System.” His expert knowledge in database management and constant motivation helped me understand advanced concepts such as normalization, transaction handling, concurrency control, and security implementation in a practical manner.

I would also like to extend my gratitude to the management and faculty members of R. K. Talreja College for providing the necessary resources, infrastructure, and academic environment required to complete this project successfully. Their encouragement and institutional support played a significant role in achieving the project objectives.

I am thankful to my classmates and friends for their cooperation, idea sharing, and moral support during the project work. Their discussions and feedback helped improve the quality and presentation of the database design and implementation.

Finally, I would like to express my heartfelt appreciation to my family for their constant encouragement, patience, and support throughout the academic year. Without their motivation and understanding, the successful completion of this project would not have been possible.

This project has enhanced my practical knowledge of MySQL and relational database concepts, and I sincerely thank everyone who directly or indirectly contributed to its completion.

INDEX

Sr. No.	Chapter Title	PAGE NO
1	Introduction	1
2	Problem Definition	2
3	Objectives of the Project	3
4	Scope of the Project	4
5	Requirement Specification	5
6	System Design	6
7	Database Design	7
8	UML Diagrams	11
9	SQL Implementation	15
10	System Testing and Result	19
11	Security, Backup and Recovery	22
12	Future Scope and Conclusion	23
13	References	23
14	Glossary	24

1. INTRODUCTION

A database system is a structured method of storing, organizing, and managing data in an efficient and reliable manner. It allows information to be stored in relational tables so that data can be easily accessed, updated, and maintained with accuracy. Database systems help reduce manual work, eliminate redundancy, maintain consistency, and improve overall data management. In educational institutions, managing student records efficiently is essential for maintaining academic discipline and transparency.

The Student Attendance Management System is a database-driven system designed to automate and manage student attendance records using MySQL. In many colleges and educational institutions, attendance is still recorded manually using registers or spreadsheets. This manual process often leads to errors such as duplicate entries, incorrect calculations of attendance percentage, misplaced records, slow data retrieval, and difficulty in generating reports. Additionally, when multiple faculty members handle attendance simultaneously, inconsistencies and data conflicts may occur.

This project solves these issues by implementing a structured relational database system. All attendance-related data such as student information, faculty details, subject allocation, enrollment records, and daily attendance are stored in properly related tables. Each entity is uniquely identified using primary keys, and relationships are maintained using foreign keys. The system is normalized up to Third Normal Form (3NF), which reduces redundancy and ensures consistency of stored data.

The system also implements important database concepts such as constraints, indexing, transactions, and concurrency control. Transaction management ensures that bulk attendance operations are completed fully and correctly using COMMIT and ROLLBACK commands, preventing partial data storage. Concurrency control mechanisms help manage simultaneous access by multiple faculty members, maintaining data integrity through isolation levels and row-level locking.

MySQL Workbench is used as the interface for executing SQL queries and managing the database. MySQL is selected because it is open-source, reliable, secure, and widely used in real-world database applications. It supports standard SQL commands, ACID properties, role-based access control, backup utilities, and strong data integrity features. Being open source, it is cost-effective and suitable for academic projects.

In conclusion, the Student Attendance Management System demonstrates how a relational database can be used to automate attendance tracking efficiently. The project not only reduces paperwork and human errors but also provides practical exposure to advanced DBMS concepts such as normalization, transaction handling, concurrency control, security, and backup management. This makes the project academically strong and suitable for SY BScIT level implementation.

2. PROBLEM DEFINITION

Many colleges and educational institutions still use manual registers or basic spreadsheet systems to record student attendance. These traditional methods are time-consuming, prone to human errors, and difficult to maintain as the number of students and subjects increases. Manual systems require physical record keeping, which increases paperwork and makes data retrieval slow and inefficient.
Due to the absence of a structured database system, several problems arise:

- **Data Redundancy:** Student and subject details may be entered repeatedly in different records.
- **Data Inconsistency:** Changes made in one record may not be updated everywhere, leading to conflicting information.
- **Duplicate Attendance Entries:** Attendance may be accidentally recorded more than once for the same student and date.
- **Calculation Errors:** Manual percentage calculations of attendance management may be incorrect.
- **Slow Report Generation:** Preparing attendance summaries consumes more time than sufficient time.
- **Partial data updates:** During bulk entry, if an error occurs, some records may be saved while others are not.
- **Concurrency Issues:** When multiple faculty members update attendance simultaneously, conflicts may occur.
- **Poor data security:** Manual or semi-digital systems lack proper access control and protection.
- **Risk of data Loss:** Physical registers may be damaged or misplaced.
- **Lack of Data Integrity:** Manual systems do not enforce constraints such as primary key or foreign keys.
- **Limited reporting Capabilities:** Manual system cannot easily generate advanced reports such as subject-wise attendance percentage, faculty wise reports, or monthly summaries without additional efforts

These issues highlight the need for a secure and structured D**atabase-driven Student Attendance Management System**. By using MySQL, attendance data can be managed efficiently with proper constraints, transaction handling, and concurrency control, ensuring accuracy, consistency, and reliability.

3. OBJECTIVES OF THE PROJECT

The **Student Attendance Management System** is developed with the aim of creating a structured and reliable database solution for managing student attendance records efficiently. The system focuses on replacing traditional manual attendance registers with a relational database that ensures accuracy, consistency, and fast retrieval of information.

The main objectives of the Student Attendance Management System are as follows:

- To design and develop a structured database system for managing attendance using MySQL.
- To create and maintain related tables such as Student, Faculty, Subject, Enrollment, and Attendance.
- To implement SQL queries for insertion, updating, deletion, and retrieval of attendance records.
- To apply database constraints such as Primary Key, Foreign Key, NOT NULL, UNIQUE, and CHECK to ensure data integrity.
- To normalize the database up to Third Normal Form (3NF) to reduce redundancy.
- To prevent duplicate attendance entries using validation rules.
- To implement transaction management for bulk attendance operations using COMMIT and ROLLBACK.
- To handle concurrent access by multiple users using isolation levels and locking mechanisms.
- To generate accurate and meaningful attendance reports using SQL queries.
- To gain practical exposure to advanced DBMS concepts using MySQL Workbench.

Overall, the objective of this project is to develop an efficient, secure, and reliable attendance management system while strengthening practical knowledge of relational database design and implementation.

4. SCOPE OF THE PROJECT

The Student Attendance Management System is designed to manage student attendance records using a structured database-driven approach. The system focuses on organizing student details, subject allocation, faculty information, enrollment records, and daily attendance entries in a relational database. It is suitable for colleges and small educational institutions where attendance needs to be recorded accurately and efficiently.

Users of the System

- **Administrator:** Responsible for managing student records, faculty details, subject creation, enrollment management, and database maintenance. The administrator also controls user access and permissions.
- **Faculty Members:** Responsible for marking daily attendance, updating attendance status, and viewing attendance reports for their respective subjects.
- **Educational Institutions:** Colleges and small institutions can use the system to maintain structured attendance records efficiently.
- **Students (Academic Learning Purpose):** Students of Information Technology can use this system to understand practical implementation of database concepts such as normalization, transactions, and concurrency control.

Areas of Use

- **Educational Use:**
  The system is designed mainly for academic projects and practical learning. It demonstrates how relational databases can be used to manage attendance efficiently.
- **Institutional Use:**
  Small and medium-scale colleges can use the system to manage daily attendance records, generate reports, and maintain organized data storage.
- **Administrative Use:**
  Administrators can maintain and monitor attendance data systematically, reducing manual record-keeping efforts.

Academic Limitations

- The system is primarily designed for academic and small-scale implementation.
- It does not include advanced features such as full web-based deployment, biometric attendance integration, or mobile application support.
- The main focus of the project is on database design and DBMS concepts rather than large-scale application development.

Overall, the Student Attendance Management System provides a structured and reliable solution for managing attendance records using MySQ

5. REQUIREMENT SPECIFICATION

5.1 Hardware Requirements

The following hardware requirements are necessary to run the Student Attendance Management System:

- **Computer / Laptop:**
  A standard desktop computer or laptop capable of running MySQL Server and related database tools.

- **Minimum RAM:**
  At least **4 GB RAM** is required for smooth functioning of the MySQL Workbench and query execution..

- **Storage:**
  Minimum 10 GB free disk space is required to install the database software, store attendance records, and backup files.

5.2 Software Requirements

The following software tools are required for the development and execution of the Student Attendance Management System:

Software	Purpose
MySQL	Used to create, store, and manage the attendance database
MySQL Workbench	Provides graphical interface for writing and executing SQL queries.
SQL	Query language used to perform database operations such as INSERT, UPDATE, DELETE, and SELECT
Windows/Linux Operating System	Platform required to install and run MySQL and related tools
Mysqldump Utility	Used for database backup and recovery

6. SYSTEM DESIGN

6.1 System Architecture (Conceptual)

The Student Attendance Management System follows a database-centered architecture where users interact with the system through SQL queries executed on a MySQL database. The architecture ensures proper data organization, data integrity, and reliable transaction handling.

In this system, users such as the Administrator and Faculty members access the database using SQL commands through MySQL Workbench or a similar query interface. They perform operations like student registration, subject allocation, enrollment management, attendance marking, and report generation by executing SQL queries.

The MySQL Server acts as the core component of the system. It stores all attendance-related data in structured tables such as Student, Faculty, Subject, Enrollment, and Attendance. The server enforces constraints like Primary Keys and Foreign Keys to maintain data integrity. It also manages transactions to ensure that bulk attendance updates are completed fully and correctly.

Whenever a user executes a query, the request is sent to the MySQL Server for processing. The server checks query syntax, validates constraints, verifies user privileges, and then performs the required operation. If the query involves multiple steps, transaction control ensures that either all changes are saved (COMMIT) or none are applied (ROLLBACK). The result is then returned to the user.

Overall, this conceptual system design provides a simple and reliable structure for managing attendance records efficiently while demonstrating practical implementation of database concepts such as constraints, normalization, transactions, and concurrency control.

7. DATABASE DESIGN

The database design of the Student Attendance Management System is based on the relational model. The entire system is divided into multiple related tables to store student, faculty, subject, enrollment, and attendance data in a structured manner. Each table represents a real-world entity, and relationships between tables are maintained using primary and foreign keys to ensure data integrity and consistency.

7.1 Entity Description

Student Table
The Student table stores information related to students. It includes details such as roll number, name, email, and department. Each student is uniquely identified using a student_id. This table helps maintain unique student records and avoids duplication.

Faculty Table
The Faculty table contains information about faculty members such as faculty name, email, and department. Each faculty member is uniquely identified using faculty_id. This table helps in assigning subjects to faculty members.

Subject Table
The Subject table stores subject-related information including subject name and the faculty assigned to teach it. Each subject is uniquely identified using subject_id and is linked to the Faculty table using a foreign key.

Enrollment Table
The Enrollment table establishes a relationship between students and subjects. It records which student is enrolled in which subject. This table supports the many-to-many relationship between Student and Subject.

Attendance Table
The Attendance table stores daily attendance records. It includes enrollment_id, date, and attendance status (Present or Absent). This table helps track subject-wise attendance of each student.

7.2 Table Structure

Student Table

Attribute	Data Type
student_id	INT (Primary Key, Auto Increment)
roll_no	VARCHAR(20) (UNIQUE)
name	VARCHAR(15) (NOT NULL)
email	VARCHAR(50) (UNIQUE)
department	VARCHAR(50)

Faculty Table

Attribute	Data Type
faculty_id	INT (Primary Key, Auto Increment)
name	VARCHAR(50) (NOT NULL)
email	VARCHAR(50) (UNIQUE)
department	VARCHAR(50)

Subject Table

Attribute	Data Type
subject_id	INT (Primary Key, Auto Increment)
subject_name	VARCHAR(50) (NOT NULL)
Faculty_id	INT (FOREIGN KEY)

Enrollment Table

Attribute	Data Type
enrollment_id	INT (Primary Key, Auto Increment)
student_id	INT (Foreign Key)
subject_id	INT (Foreign Key)

Attendance Table

Attribute	Data Type
attendance_id	INT (Primary Key, Auto Increment)
enrollment_id	INT (Foreign Key)
date	DATE
status	VARCHAR(10) CHECK (status IN (‘Present’Absent’))

7.3 Constraints Used

Constraints are applied to maintain accuracy and consistency in the database.

PRIMARY KEY

The PRIMARY KEY constraint is used to uniquely identify each record in every table.

FOREIGN KEY

The FOREIGN KEY constraint is used to maintains relationships between tables such as Student–Enrollment and Enrollment–Attendance.

NOT NULL

The NOT NULL constraint is used to ensure ensures that essential fields like student name and subject name are not left empty.

UNIQUE

The UNIQUE constraint is used to Prevents duplicate values such as roll number and email addresses.

CHECK Constraint

The CHECK constraints ensures attendance status can only be ‘Present’ or ‘Absent’.

8. UML DIAGRAMS

In this project, various UML diagrams are used to represent the structure and working of the **Student Attendance Management System**. These diagrams help in understanding the database design, system workflow, and interaction between users and the database.

8.1 ER Diagram

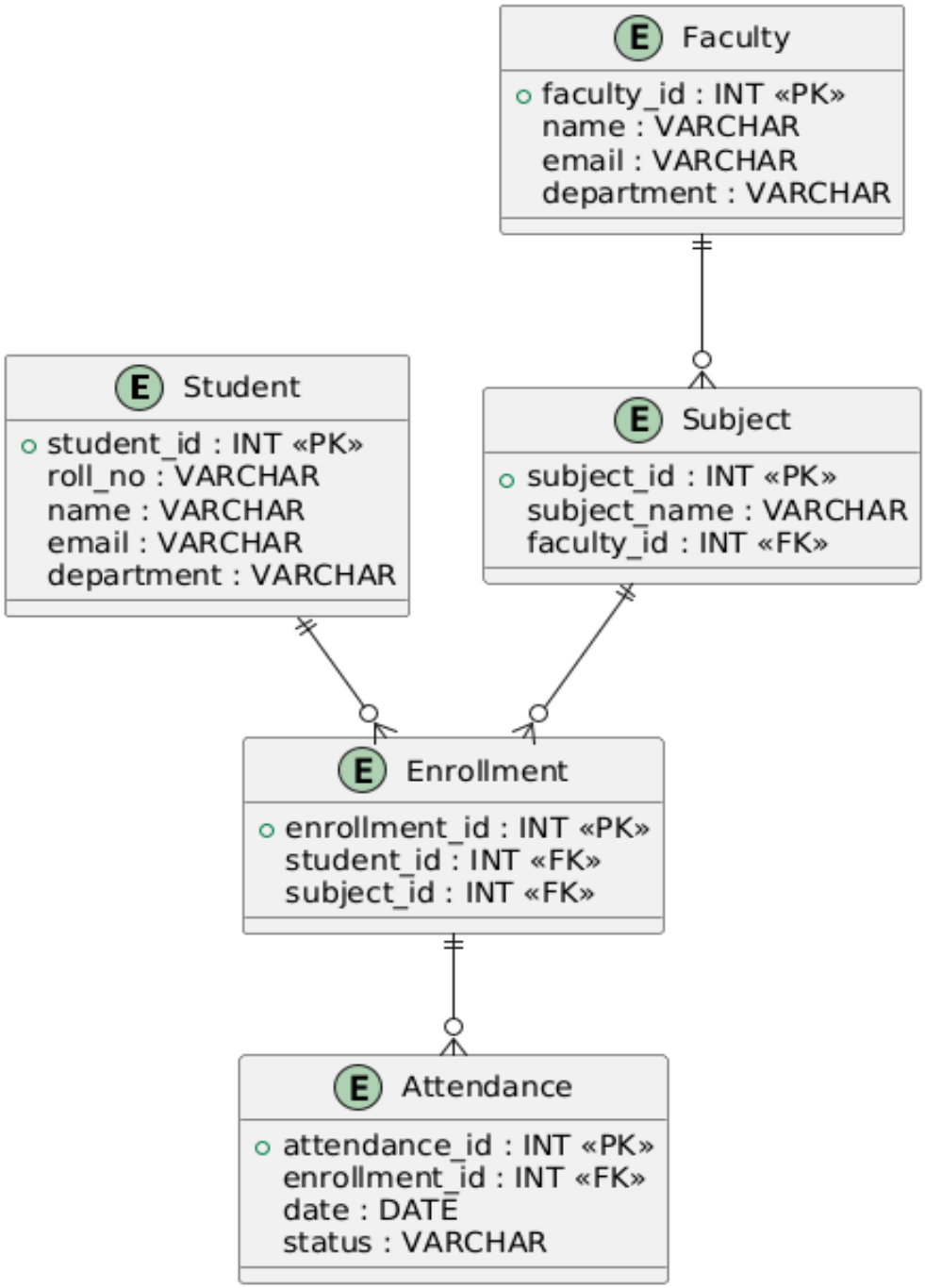

Figure 8.1: ER Diagram

• The ER diagram shows the structural design of the Attendance Management System database.
• It contains five main entities: Student, Faculty, Subject, Enrollment, and Attendance.
• Enrollment acts as a bridge between Student and Subject, handling the many-to-many relationship.
• Attendance is linked to Enrollment, ensuring attendance is recorded only for enrolled students.
• Primary keys uniquely identify records, and foreign keys maintain referential integrity.

8.2 Use Case Diagram

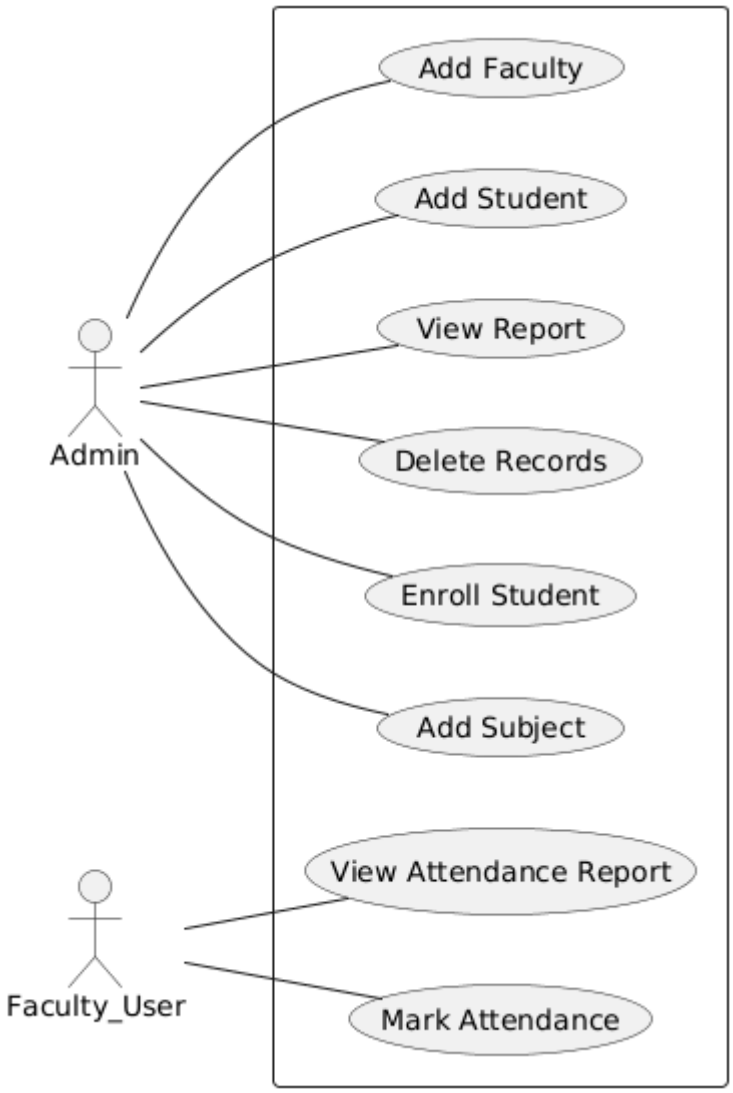

Figure 8.2: Use Case Diagram

• The Use Case diagram represents how users interact with the system.
• There are two actors: Admin and Faculty User.
• Admin manages students, faculty, subjects, enrollments, and reports.
• Faculty User mainly marks attendance and views attendance reports.
• It shows system functionality from a user perspective, not internal database logic.

8.3 Activity Diagram

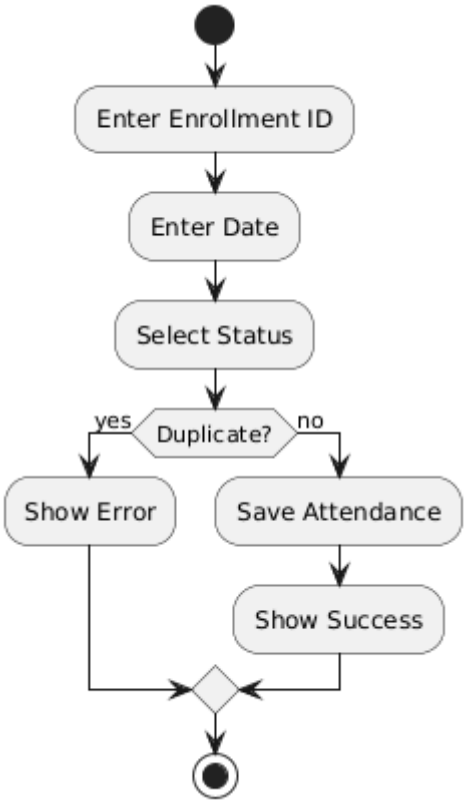

Figure 8.3: Activity Diagram

• The Activity diagram represents the workflow of the attendance process.
• The process starts when enrollment ID, date, and status are entered.
• The system checks whether duplicate attendance exists.
• If duplicate is found, an error is shown; otherwise, attendance is stored.
• The process ends after successful insertion or error notification.

8.4 Sequence Diagram

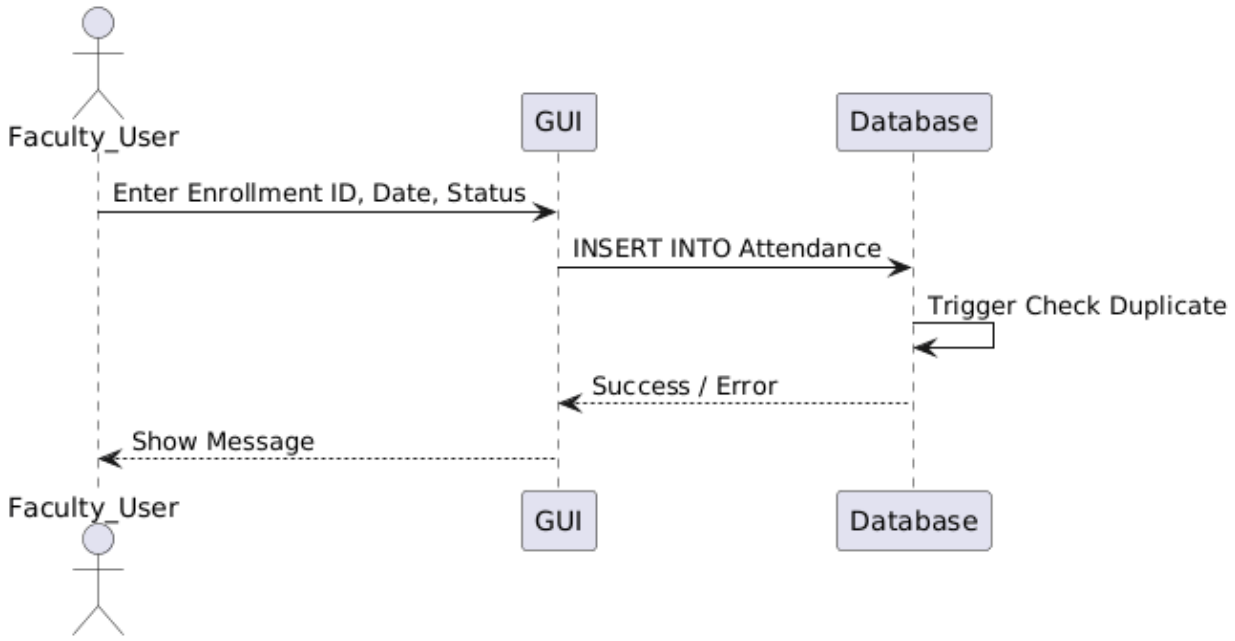

Figure 8.4: Sequence Diagram

• The Sequence diagram shows the step-by-step interaction during attendance marking.
• Faculty enters data in the GUI.
• GUI sends an insert request to the database.
• The database trigger checks for duplicate attendance.
• System returns either a success message or an error response.

9. SQL IMPLEMENTATION

This section explains the SQL commands used to implement the Student Attendance Management System. SQL (Structured Query Language) is used to create the database, define table structures, insert data, manage transactions, create procedures and triggers, and retrieve attendance reports.
The project is implemented using **MySQL** and executed through **MySQL Workbench**.

9.1 Database Creation

The database is first created using the CREATE DATABASE command. After creation, it is selected using the USE command.
Example:

```
CREATE DATABASE attendance_db;
USE attendance_db;
```

9.2 Table Creation

Tables are created using the CREATE TABLE command along with appropriate data types and constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and cascading rules.
Example (Student Table):

```
CREATE TABLE Student (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    roll_no VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    department VARCHAR(50) NOT NULL
);
```

The Attendance table includes:

- CHECK constraint for status ('Present','Absent')
- UNIQUE(enrollment_id, date) to prevent duplicate attendance

9.3 Data Insertion

The INSERT INTO command is used to add records into the tables.
Example:

```
INSERT INTO Student (roll_no, name, email, department)
VALUES ('IT101', 'Rahul Sharma', 'rahul@email.com', 'IT');
```

9.4 Data Retrieval (SELECT with JOIN)

The SELECT statement is used to retrieve meaningful attendance reports by joining multiple tables.
Example:

```
SELECT
s.roll_no,
s.name,
sub.subject_name,
a.date,
a.status
FROM Attendance a
JOIN Enrollment e ON a.enrollment_id = e.enrollment_id
JOIN Student s ON e.student_id = s.student_id
JOIN Subject sub ON e.subject_id = sub.subject_id;
```

9.5 Advanced Queries

Advanced subqueries are used to perform complex data retrieval operations by embedding one query inside another. These subqueries help in filtering results, performing aggregations, and generating meaningful reports based on related tables.

GROUP BY and HAVING

The following query finds students who have more than one attendance record.

```
SELECT name
FROM Student
WHERE student_id IN (
    SELECT e.student_id
    FROM Enrollment e
    JOIN Attendance a
    ON e.enrollment_id = a.enrollment_id
    GROUP BY e.student_id
    HAVING COUNT(a.attendance_id) > 1
);
```

This subquery uses aggregation to filter grouped results.

SUBQUERIES

Subqueries are queries written inside another SQL query. In this project, The EXIST operator is used for checks whther related records exist .

Example:

```
SELECT 1
    FROM Enrollment e
    JOIN Attendance a
    ON e.enrollment_id = a.enrollment_id
    WHERE e.student_id = s.student_id
```

VIEWS

This SQL statement creates a view named `Attendance_Report` to display a complete attendance summary.
It joins five tables: Attendance, Enrollment, Student, Subject, and Faculty to combine related data.
The view shows student details, faculty name, subject name, attendance date, and status in one structured result.
It simplifies complex joins, so you can retrieve the full attendance report using a simple `SELECT * FROM Attendance_Report;`.

Example:

```
-- VIEW (FINAL REPORT)
CREATE VIEW Attendance_Report AS
SELECT
s.student_id,
s.roll_no,
s.name AS student_name,
s.department,
f.name AS faculty_name,
sub.subject_name,
a.date,
a.status
FROM Attendance a
JOIN Enrollment e ON a.enrollment_id=e.enrollment_id
JOIN Student s ON e.student_id=s.student_id
JOIN Subject sub ON e.subject_id=sub.subject_id
JOIN Faculty f ON sub.faculty_id=f.faculty_id;
```

9.6 Transaction Management

In this project, transactions are used while inserting attendance records for multiple students.
The process begins with starting a transaction, performing attendance insert operations, and finally committing the transaction.
If any error occurs during insertion, the system can roll back the changes to maintain data integrity.

The following transaction commands are used in the project:

- START TRANSACTION

- COMMIT
- ROLLBACK

Transaction management ensures atomicity and maintains data integrity, making the system reliable and consistent.

Example:

```
START TRANSACTION;
INSERT INTO Attendance (enrollment_id,date,status)
VALUES (1,'2026-02-18','Present');
INSERT INTO Attendance (enrollment_id,date,status)
VALUES (2,'2026-02-18','Present');
COMMIT;
```

Time Distribution and Work Allocation

The project was divided into structured phases to ensure proper planning and execution. Each activity was allocated time based on its complexity and importance.
The graph below shows the number of days spent on each phase, highlighting that the SQL implementation required the most time among the activities.

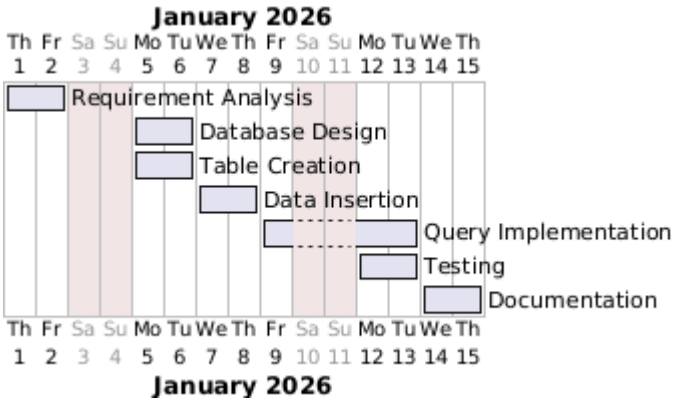

Figure 9.1: Gantt Diagram

10. SYSTEM TESTING AND RESULT

System testing was carried out to ensure that the Student Attendance Management System works correctly and produces accurate results. Different SQL queries were executed and verified to check the correctness of operations, data validity, and final output.

10.1 Query Correctness Testing

- All SQL queries were tested to ensure they return accurate results.
- SELECT * FROM Attendance_Report; was executed to verify proper JOIN operations.
- INSERT statements were tested to confirm data is stored correctly.
- Duplicate attendance was tested and the trigger correctly blocked repeated entries.
- Stored Procedure MarkAttendance was executed successfully.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Tables_in_attendance_db
attendance
attendance_report
enrollment
faculty
student
subject

Attendance Table:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

attendance_id	enrollment_id	date	status
1	1	2026-02-17	Present
2	2	2026-02-17	Absent
3	4	2026-02-01	Present
5	6	2026-02-23	Abs Absent
NULL	NULL	NULL	NULL

Student Table:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	roll_no	name	email	department
1	IT101	Rahul Sharma	rahul@email.com	IT Delete selected rows
2	IT102	Priya Singh	priya@email.com	IT
10	IT103	Urmila Badve	u@gmail.com	IT
12	IT104	Avantika Ghase	av@gmail.com	IT
NULL	NULL	NULL	NULL	NULL

Faculty Table:

faculty_id	name	email	department
1	Dr. Mehta	mehta@email.com	IT
2	Mr. Sahil Shukla	ss@gmail.com	IT
5	Mrs. Trupti Dongade	td@gmail.com	CS
7	Mr. Omkar kulkarni	ok@gmail.com	CS
NULL	NULL	NULL	NULL

Subject Table:

subject_id	subject_name	faculty_id
1	Database Management	1
2	Computer networks	5
3	Java Prog	1
NULL	NULL	NULL

Enrollment Table:

enrollment_id	student_id	subject_id
1	1	1
2	2	1
4	10	3
6	12	2
NULL	NULL	NULL

Attendance_report Table:

student_id	roll_no	student_name	department	faculty_name	subject_name	date	status
1	IT101	Rahul Sharma	IT	Dr. Mehta	Database Management	2026-02-17	Present
2	IT102	Priya Singh	IT	Dr. Mehta	Database Management	2026-02-17	Absent
10	IT103	Urmila Badve	IT	Dr. Mehta	Java Prog	2026-02-01	Present
12	IT104	Avantika Ghase	IT	Mrs. Trupti Dongade	Computer networks	2026-02-23	Absent

10.2 Data Validation

Procedure Statement:
The stored procedure was displayed using:
SHOW CREATE PROCEDURE MarkAttendance;

Procedure	sql_mode	Create Procedure	character_set_client	collation_connection	Database Collation
MarkAttendance	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	CREATE DEFINER=`janhavi`@`%` PROCEDU...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900_ai_ci

Trigger Statement:
The trigger was displayed using:
SHOW CREATE TRIGGER prevent_duplicate_attendance;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Trigger	Event	Table	Statement	Timing	Created	sql_mode	De
prevent_duplicate_attendance	INSERT	attendance	BEGIN IF EXISTS(SELECT 1 FROM Attendance ...	BEFORE	2026-02-21 13:44:47.68	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	jan

prevent_duplicate_attendance

10.3 Output Verification

Verification Points:

- Student details match inserted records.
- Faculty and subject mapping is correct.
- Attendance dates and status display accurately.

Result: Output is accurate, consistent, and reliable.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

student_id	roll_no	student_name	department	faculty_name	subject_name	date	status
1	IT101	Rahul Sharma	IT	Dr. Mehta	Database Management	2026-02-17	Present
2	IT102	Priya Singh	IT	Dr. Mehta	Database Management	2026-02-17	Absent
10	IT103	Urmila Badve	IT	Dr. Mehta	Java Prog	2026-02-01	Present
12	IT104	Avantika Ghase	IT	Mrs. Trupti Dongade	Computer networks	2026-02-23	Absent

10.4 Result Analysis

The results obtained from system testing show that the database operates correctly and efficiently. All SQL queries executed without errors, and the system successfully stored customer data, processed orders, and generated accurate bills. The validation tests confirmed that duplicate and invalid data entries were restricted. Overall, the outputs match the expected results, proving that the system meets the project requirements.

11. SECURITY, BACKUP AND RECOVERY

11.1 Security

Security is an important aspect of any database system. It ensures that data is protected from unauthorized access, misuse, or accidental modification. In the Student Attendance Management System, security is maintained through proper authentication, authorization, and data integrity controls.

11.2 Backup

Backup is the process of creating a copy of the database to protect against data loss. Data loss can occur due to hardware failure, system crashes, accidental deletion, or malicious attacks.
In the Student Attendance Management System, backups are taken periodically to ensure that all student and attendance data can be restored if needed. Regular backups help in maintaining data continuity and institutional records without disruption.

11.3 Recovery

Recovery is the process of restoring the database after failure or data loss. If the system crashes or records are accidentally deleted, the backup file can be used to rebuild the database to its previous stable state.
Recovery mechanisms ensure:

- Restoration of lost data
- Continuity of operations
- Prevention of academic record loss
- System reliability and stability

12. FUTURE SCOPE AND CONCLUSION

12.1 Future Scope

The Student Attendance Management System developed in this project fulfills the basic requirements of managing student, subject, faculty, and attendance records efficiently. However, the system can be further enhanced in several ways to improve functionality, usability, and scalability.

Example's of future scope:
1. Attendance Percentage Calculation
2. SMS/Email Notifications
3. Mobile Apllication
4.

12.2 Conclusion

The Student Attendance Management System is designed to efficiently manage and maintain attendance records of students in an organized and structured manner. The system successfully implements core database concepts such as table creation, relationships, constraints, transactions, procedures, triggers, indexing, and user access control.
By using a relational database approach, the system ensures data integrity, consistency, and reliability. It minimizes manual errors, prevents duplicate attendance entries, and maintains accurate academic records.

13. REFERENCES

1. MySQL Official Documentation, Oracle Corporation.
2. Prescribed Database Management System (DBMS) textbooks.
3. Online learning resources related to SQL and MySQL.

14. GLOSSARY

- DBMS (Database Management System):
  Software used to store, manage, and retrieve data efficiently in the form of databases.
- SQL (Structured Query Language):
  A standard language used to create, insert, update, delete, and retrieve data from a database.
- Primary Key:
  A unique identifier for each record in a table that does not allow duplicate or NULL values.
- Foreign Key:
  A field in a table that creates a relationship with the primary key of another table to maintain data consistency.
- MySQL:
  An open-source relational database management system used to store and manage structured data.

15. APPENDIX / SQL CODE

```
CREATE DATABASE attendance_db;
USE attendance_db;

-- STUDENT TABLE
CREATE TABLE Student (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    roll_no VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    department VARCHAR(50) NOT NULL
);

-- FACULTY TABLE
CREATE TABLE Faculty (
    faculty_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    department VARCHAR(50) NOT NULL
);

-- SUBJECT TABLE
CREATE TABLE Subject (
    subject_id INT AUTO_INCREMENT PRIMARY KEY,
    subject_name VARCHAR(50) NOT NULL,
    faculty_id INT,
    FOREIGN KEY (faculty_id)
    REFERENCES Faculty(faculty_id)
);

-- ENROLLMENT TABLE
CREATE TABLE Enrollment (
```

```sql
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    subject_id INT NOT NULL,
    FOREIGN KEY (student_id)
    REFERENCES Student(student_id),
    FOREIGN KEY (subject_id)
    REFERENCES Subject(subject_id),
    UNIQUE(student_id, subject_id)
);

-- ATTENDANCE TABLE
CREATE TABLE Attendance (
    attendance_id INT AUTO_INCREMENT PRIMARY KEY,
    enrollment_id INT NOT NULL,
    date DATE NOT NULL,
    status VARCHAR(10) NOT NULL,
    FOREIGN KEY (enrollment_id)
    REFERENCES Enrollment(enrollment_id),
    CHECK (status IN ('Present','Absent')),
    UNIQUE(enrollment_id, date)
);

-- INSERT DATA
INSERT INTO Student (roll_no, name, email, department)
VALUES
('IT101','Rahul Sharma','rahul@email.com','IT'),
('IT102','Priya Singh','priya@email.com','IT');

INSERT INTO Faculty (name, email, department)
VALUES
('Dr. Mehta','mehta@email.com','IT');

INSERT INTO Subject (subject_name, faculty_id)
VALUES
('Database Management',1);

INSERT INTO Enrollment (student_id, subject_id)
VALUES
(1,1),
(2,1);

INSERT INTO Attendance (enrollment_id, date, status)
VALUES
(1,'2026-02-17','Present'),
(2,'2026-02-17','Absent');

-- TRANSACTION
START TRANSACTION;

INSERT INTO Attendance (enrollment_id,date,status)
VALUES (1,'2026-02-18','Present');
```

```
INSERT INTO Attendance (enrollment_id,date,status)
VALUES (2,'2026-02-18','Present');

COMMIT;

-- STORED PROCEDURE
DELIMITER //

CREATE PROCEDURE MarkAttendance(
IN enroll_id INT,
IN att_date DATE,
IN att_status VARCHAR(10)
)
BEGIN
INSERT INTO Attendance(enrollment_id,date,status)
VALUES(enroll_id,att_date,att_status);
END //

DELIMITER ;

CALL MarkAttendance(1,'2026-02-19','Present');

-- TRIGGER
DELIMITER //

CREATE TRIGGER prevent_duplicate_attendance
BEFORE INSERT ON Attendance
FOR EACH ROW
BEGIN
IF EXISTS(
SELECT 1 FROM Attendance
WHERE enrollment_id=NEW.enrollment_id
AND date=NEW.date
)
THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT='Duplicate attendance not allowed';
END IF;
END //

DELIMITER ;

-- INDEX
CREATE INDEX idx_roll_no ON Student(roll_no);

-- USER SECURITY
CREATE USER 'faculty_user'@'localhost'
IDENTIFIED BY 'faculty123';

GRANT SELECT, INSERT ON attendance_db.Attendance
```

```
TO 'faculty_user'@'localhost';

-- VIEW (FINAL REPORT)
CREATE VIEW Attendance_Report AS
SELECT
s.student_id,
s.roll_no,
s.name AS student_name,
s.department,
f.name AS faculty_name,
sub.subject_name,
a.date,
a.status
FROM Attendance a
JOIN Enrollment e ON a.enrollment_id=e.enrollment_id
JOIN Student s ON e.student_id=s.student_id
JOIN Subject sub ON e.subject_id=sub.subject_id
JOIN Faculty f ON sub.faculty_id=f.faculty_id;

-- FINAL OUTPUT
SELECT * FROM Attendance_Report;
```

Python Tkinter (GUI)

```
import tkinter as tk
from tkinter import ttk, messagebox
import mysql.connector

# ---------------- DATABASE CONNECTION ----------------
def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="janhavi",
        password="root_456_j",  # CHANGE IF NEEDED
        database="attendance_db"
    )

# ---------------- MAIN WINDOW ----------------
root = tk.Tk()
root.title("Attendance Management System")
root.geometry("1250x700")
root.configure(bg="#f0f2f5")

title = tk.Label(root, text="Attendance Management System",
                 font=("Segoe UI", 22, "bold"),
                 bg="#1f2937", fg="white", pady=12)
title.pack(fill="x")

notebook = ttk.Notebook(root)
notebook.pack(fill="both", expand=True)

# ---------------- COMMON TREE FUNCTION ----------------
def create_tree(frame, columns):
    tree = ttk.Treeview(frame, columns=columns, show="headings")
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=150)
    tree.pack(fill="both", expand=True, pady=10)
    return tree

# ========================================================
# ================= STUDENT TAB ========================
# ========================================================
student_tab = tk.Frame(notebook, bg="white")
notebook.add(student_tab, text="Student")

tk.Label(student_tab, text="Roll No").pack()
roll_entry = tk.Entry(student_tab)
roll_entry.pack()

tk.Label(student_tab, text="Name").pack()
name_entry = tk.Entry(student_tab)
```

```
name_entry.pack()

tk.Label(student_tab, text="Email").pack()
email_entry = tk.Entry(student_tab)
email_entry.pack()

tk.Label(student_tab, text="Department").pack()
dept_entry = tk.Entry(student_tab)
dept_entry.pack()

def add_student():
    try:
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("INSERT INTO Student(roll_no,name,email,department)
VALUES(%s,%s,%s,%s)",
                    (roll_entry.get(), name_entry.get(), email_entry.get(),
dept_entry.get()))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", "Student Added")
        view_students()
        load_students()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def delete_student():
    selected = student_tree.focus()
    if not selected:
        messagebox.showwarning("Warning", "Select a student to delete")
        return

    data = student_tree.item(selected)["values"]
    student_id = data[0]

    try:
        conn = get_connection()
        cur = conn.cursor()

        # Delete attendance linked to this student
        cur.execute("""
            DELETE FROM Attendance
            WHERE enrollment_id IN
            (SELECT enrollment_id FROM Enrollment WHERE student_id=%s)
        """, (student_id,))

        # Delete enrollment
        cur.execute("DELETE FROM Enrollment WHERE student_id=%s",
(student_id,))

        # Delete student
```

```
        cur.execute("DELETE FROM Student WHERE student_id=%s",
(student_id,))

        conn.commit()
        conn.close()

        messagebox.showinfo("Success", "Student Deleted Successfully")
        view_students()
        load_students()
        view_report()

    except Exception as e:
        messagebox.showerror("Error", str(e))

def view_students():
    for row in student_tree.get_children():
        student_tree.delete(row)
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Student")
    for row in cur.fetchall():
        student_tree.insert("", "end", values=row)
    conn.close()

tk.Button(student_tab, text="Add Student", bg="#16a34a",
          fg="white", command=add_student).pack(pady=5)

tk.Button(student_tab, text="Delete Student", bg="#dc2626",
          fg="white", command=delete_student).pack(pady=5)

student_tree = create_tree(student_tab,
                           ("Student ID","Roll
No","Name","Email","Department"))
view_students()


# ========================================================
# ================= FACULTY TAB ========================
# ========================================================
faculty_tab = tk.Frame(notebook, bg="white")
notebook.add(faculty_tab, text="Faculty")

tk.Label(faculty_tab, text="Name").pack()
fac_name = tk.Entry(faculty_tab)
fac_name.pack()

tk.Label(faculty_tab, text="Email").pack()
fac_email = tk.Entry(faculty_tab)
fac_email.pack()

tk.Label(faculty_tab, text="Department").pack()
```

```
fac_dept = tk.Entry(faculty_tab)
fac_dept.pack()

def add_faculty():
    try:
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("INSERT INTO Faculty(name,email,department)
VALUES(%s,%s,%s)",
                    (fac_name.get(), fac_email.get(), fac_dept.get()))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", "Faculty Added")
        view_faculty()
        load_faculty()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def delete_faculty():
    selected = faculty_tree.focus()
    if not selected:
        messagebox.showwarning("Warning", "Select faculty to delete")
        return

    data = faculty_tree.item(selected)["values"]
    faculty_id = data[0]

    try:
        conn = get_connection()
        cur = conn.cursor()

        # Set faculty_id to NULL in Subject first (if allowed)
        cur.execute("UPDATE Subject SET faculty_id=NULL WHERE
faculty_id=%s", (faculty_id,))

        cur.execute("DELETE FROM Faculty WHERE faculty_id=%s",
(faculty_id,))

        conn.commit()
        conn.close()

        messagebox.showinfo("Success", "Faculty Deleted Successfully")
        view_faculty()
        load_faculty()
        view_report()

    except Exception as e:
        messagebox.showerror("Error", str(e))

def view_faculty():
    for row in faculty_tree.get_children():
```

```
        faculty_tree.delete(row)
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Faculty")
    for row in cur.fetchall():
        faculty_tree.insert("", "end", values=row)
    conn.close()

tk.Button(faculty_tab, text="Add Faculty",
          bg="#2563eb", fg="white", command=add_faculty).pack(pady=5)

tk.Button(faculty_tab, text="Delete Faculty",
          bg="#dc2626", fg="white", command=delete_faculty).pack(pady=5)

faculty_tree = create_tree(faculty_tab,
                           ("Faculty ID","Name","Email","Department"))
view_faculty()

# ========================================================
# ================= SUBJECT TAB ========================
# ========================================================
subject_tab = tk.Frame(notebook, bg="white")
notebook.add(subject_tab, text="Subject")

tk.Label(subject_tab, text="Subject Name").pack()
sub_name = tk.Entry(subject_tab)
sub_name.pack()

tk.Label(subject_tab, text="Select Faculty").pack()
faculty_combo = ttk.Combobox(subject_tab, state="readonly")
faculty_combo.pack()

def load_faculty():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT faculty_id, name FROM Faculty")
    data = cur.fetchall()
    faculty_combo['values'] = [f"{row[0]} - {row[1]}" for row in data]
    conn.close()

def add_subject():
    try:
        faculty_id = faculty_combo.get().split(" - ")[0]
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("INSERT INTO Subject(subject_name,faculty_id)
VALUES(%s,%s)",
                    (sub_name.get(), faculty_id))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", "Subject Added")
```

```
        view_subject()
        load_subjects()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def delete_subject():
    selected = subject_tree.focus()
    if not selected:
        messagebox.showwarning("Warning", "Select a subject to delete")
        return

    data = subject_tree.item(selected)["values"]
    subject_id = data[0]

    try:
        conn = get_connection()
        cur = conn.cursor()

        # Delete attendance through enrollment
        cur.execute("""
            DELETE FROM Attendance
            WHERE enrollment_id IN
            (SELECT enrollment_id FROM Enrollment WHERE subject_id=%s)
        """, (subject_id,))

        # Delete enrollment
        cur.execute("DELETE FROM Enrollment WHERE subject_id=%s",
(subject_id,))

        # Delete subject
        cur.execute("DELETE FROM Subject WHERE subject_id=%s",
(subject_id,))

        conn.commit()
        conn.close()

        messagebox.showinfo("Success", "Subject Deleted Successfully")
        view_subject()
        load_subjects()
        view_report()

    except Exception as e:
        messagebox.showerror("Error", str(e))

def view_subject():
    for row in subject_tree.get_children():
        subject_tree.delete(row)
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Subject")
    for row in cur.fetchall():
```

```
        subject_tree.insert("", "end", values=row)
    conn.close()

tk.Button(subject_tab, text="Add Subject",
          bg="#7c3aed", fg="white", command=add_subject).pack(pady=5)

tk.Button(subject_tab, text="Delete Subject",
          bg="#dc2626", fg="white", command=delete_subject).pack(pady=5)

subject_tree = create_tree(subject_tab,
                           ("Subject ID","Subject Name","Faculty ID"))
view_subject()

# ========================================================
# ================= ENROLLMENT TAB =====================
# ========================================================
enroll_tab = tk.Frame(notebook, bg="white")
notebook.add(enroll_tab, text="Enrollment")

tk.Label(enroll_tab, text="Select Student").pack()
student_combo = ttk.Combobox(enroll_tab, state="readonly")
student_combo.pack()

tk.Label(enroll_tab, text="Select Subject").pack()
subject_combo = ttk.Combobox(enroll_tab, state="readonly")
subject_combo.pack()

def load_students():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT student_id, name FROM Student")
    data = cur.fetchall()
    student_combo['values'] = [f"{row[0]} - {row[1]}" for row in data]
    conn.close()

def load_subjects():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT subject_id, subject_name FROM Subject")
    data = cur.fetchall()
    subject_combo['values'] = [f"{row[0]} - {row[1]}" for row in data]
    conn.close()

def add_enrollment():
    try:
        student_id = student_combo.get().split(" - ")[0]
        subject_id = subject_combo.get().split(" - ")[0]
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("INSERT INTO Enrollment(student_id,subject_id)
VALUES(%s,%s)",
```

```
                    (student_id, subject_id))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", "Enrollment Added")
        view_enrollment()
        load_enrollments()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def delete_enrollment():
    selected = enroll_tree.focus()
    if not selected:
        messagebox.showwarning("Warning", "Select enrollment to delete")
        return

    data = enroll_tree.item(selected)["values"]
    enrollment_id = data[0]

    try:
        conn = get_connection()
        cur = conn.cursor()

        cur.execute("DELETE FROM Attendance WHERE enrollment_id=%s",
(enrollment_id,))
        cur.execute("DELETE FROM Enrollment WHERE enrollment_id=%s",
(enrollment_id,))

        conn.commit()
        conn.close()

        messagebox.showinfo("Success", "Enrollment Deleted Successfully")
        view_enrollment()
        load_enrollments()
        view_report()

    except Exception as e:
        messagebox.showerror("Error", str(e))

def view_enrollment():
    for row in enroll_tree.get_children():
        enroll_tree.delete(row)
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Enrollment")
    for row in cur.fetchall():
        enroll_tree.insert("", "end", values=row)
    conn.close()

tk.Button(enroll_tab, text="Add Enrollment",
          bg="#10b981", fg="white", command=add_enrollment).pack(pady=5)
```

```
tk.Button(enroll_tab, text="Delete Enrollment",
          bg="#dc2626", fg="white", command=delete_enrollment).pack(pady=5)

enroll_tree = create_tree(enroll_tab,
                          ("Enrollment ID","Student ID","Subject ID"))
view_enrollment()

# ======================================================
# ================= ATTENDANCE TAB =====================
# ======================================================
attendance_tab = tk.Frame(notebook, bg="white")
notebook.add(attendance_tab, text="Attendance")

tk.Label(attendance_tab, text="Select Enrollment").pack()
enroll_combo = ttk.Combobox(attendance_tab, state="readonly")
enroll_combo.pack()

tk.Label(attendance_tab, text="Date (YYYY-MM-DD)").pack()
date_entry = tk.Entry(attendance_tab)
date_entry.pack()

tk.Label(attendance_tab, text="Status").pack()
status_combo = ttk.Combobox(attendance_tab,
                            values=["Present","Absent"],
                            state="readonly")
status_combo.pack()

def load_enrollments():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT enrollment_id FROM Enrollment")
    data = cur.fetchall()
    enroll_combo['values'] = [row[0] for row in data]
    conn.close()

def mark_attendance():
    try:
        conn = get_connection()
        cur = conn.cursor()
        cur.callproc("MarkAttendance",
                     (enroll_combo.get(), date_entry.get(),
status_combo.get()))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", "Attendance Marked")
        view_attendance()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def delete_attendance():
    selected = attendance_tree.focus()
```

```
    if not selected:
        messagebox.showwarning("Warning", "Select attendance to delete")
        return

    data = attendance_tree.item(selected)["values"]
    attendance_id = data[0]

    try:
        conn = get_connection()
        cur = conn.cursor()

        cur.execute("DELETE FROM Attendance WHERE attendance_id=%s",
(attendance_id,))

        conn.commit()
        conn.close()

        messagebox.showinfo("Success", "Attendance Deleted Successfully")
        view_attendance()
        view_report()

    except Exception as e:
        messagebox.showerror("Error", str(e))

def view_attendance():
    for row in attendance_tree.get_children():
        attendance_tree.delete(row)
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Attendance")
    for row in cur.fetchall():
        attendance_tree.insert("", "end", values=row)
    conn.close()

tk.Button(attendance_tab, text="Mark Attendance",
          bg="#0ea5e9", fg="white", command=mark_attendance).pack(pady=5)

tk.Button(attendance_tab, text="Delete Attendance",
          bg="#dc2626", fg="white", command=delete_attendance).pack(pady=5)

attendance_tree = create_tree(attendance_tab,
                              ("Attendance ID","Enrollment
ID","Date","Status"))
view_attendance()


# ---------------- INITIAL LOAD ----------------
load_students()
load_subjects()
load_faculty()
load_enrollments()
```

```
# ---------------- ATTENDANCE-VIEW TAB ------------
report_tab = tk.Frame(notebook, bg="white")
notebook.add(report_tab, text="View Attendance")

search_frame = tk.Frame(report_tab, bg="white")
search_frame.pack(pady=10)

tk.Label(search_frame, text="Search by Roll No:",
         bg="white").grid(row=0, column=0, padx=5)

search_entry = tk.Entry(search_frame)
search_entry.grid(row=0, column=1, padx=5)

report_tree = ttk.Treeview(report_tab,
                           columns=("Student ID","Roll No","Student Name",
                                    "Department","Faculty Name",
                                    "Subject","Date","Status"),
                           show="headings")

for col in report_tree["columns"]:
    report_tree.heading(col, text=col)
    report_tree.column(col, width=140)

report_tree.pack(fill="both", expand=True, pady=10)

def view_report():
    for row in report_tree.get_children():
        report_tree.delete(row)

    conn = get_connection()
    cur = conn.cursor()

    query = """
    SELECT
        s.student_id,
        s.roll_no,
        s.name,
        s.department,
        f.name,
        sub.subject_name,
        a.date,
        a.status
    FROM Student s
    LEFT JOIN Enrollment e ON s.student_id = e.student_id
    LEFT JOIN Subject sub ON e.subject_id = sub.subject_id
    LEFT JOIN Faculty f ON sub.faculty_id = f.faculty_id
    LEFT JOIN Attendance a ON e.enrollment_id = a.enrollment_id
    """

    if search_entry.get() != "":
```

```
        query += " WHERE s.roll_no = %s"
        cur.execute(query, (search_entry.get(),))
    else:
        cur.execute(query)

    for row in cur.fetchall():
        report_tree.insert("", "end", values=row)

    conn.close()

tk.Button(search_frame, text="Search",
          bg="#2563eb", fg="white",
          command=view_report).grid(row=0, column=2, padx=5)

tk.Button(search_frame, text="Show All",
          bg="#10b981", fg="white",
          command=view_report).grid(row=0, column=3, padx=5)

view_report()

root.mainloop()
```